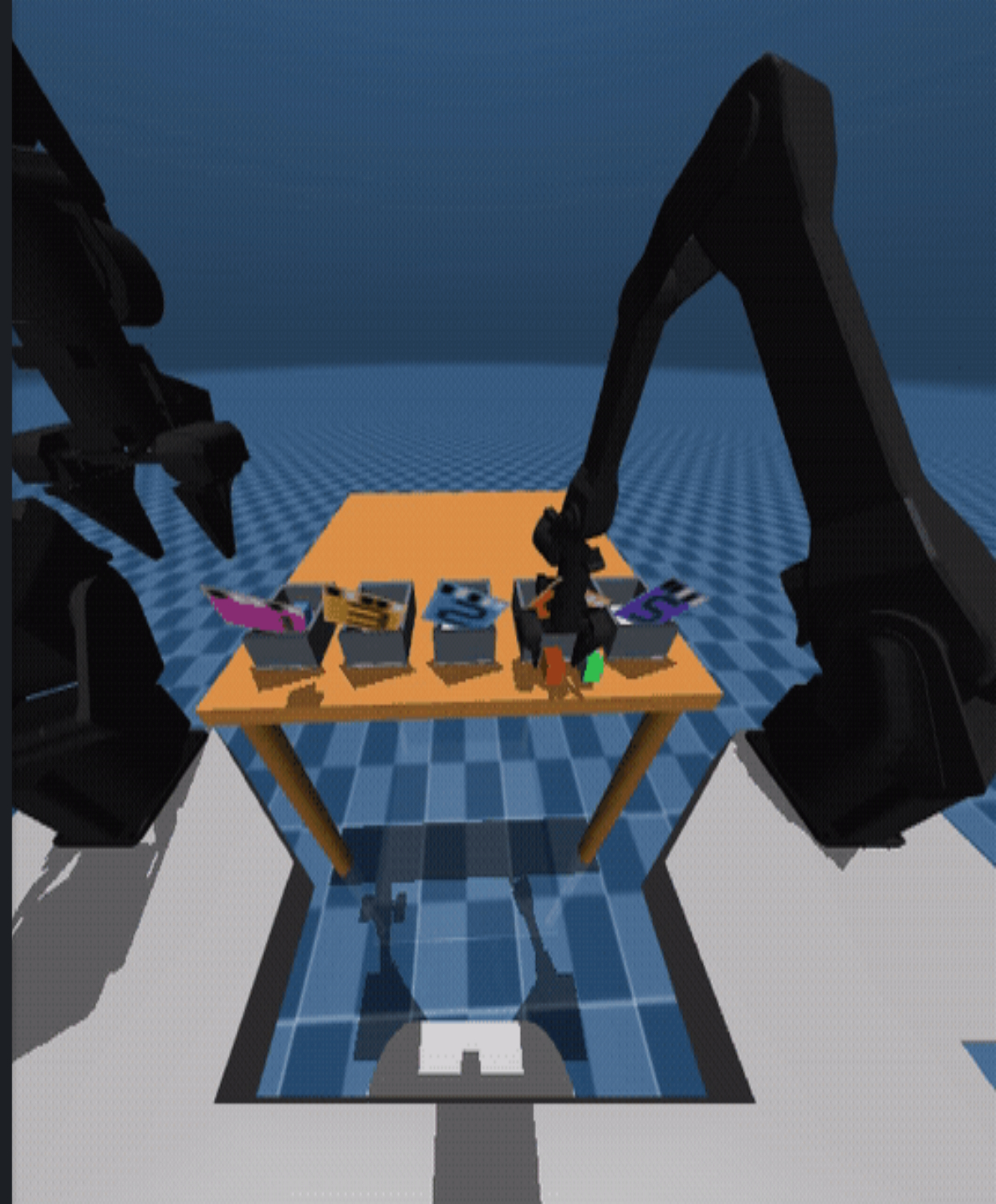


在线代码策略在动态具身任务中的优势

CaP / HoloAgent-0 Local / PhyAgentOS Local 对比实验

Trossen MuJoCo • 三相机 RGB-D
Visual TAMP • seeds 411-413

Hybrid CapX / CapGym 本地实验



具身 Agent 不是单一模型，
而是闭环系统。

语言模型负责理解任务和组合已有能力；
感知系统把图像转成可操作对象；
TAMP 处理几何可行性；控制器执行连续运动；验证器把物理结果反馈给 Agent。



关键边界：CaP 能零样本组合已有 primitive；它不能凭空创造抓取、系绳等新的底层物理技能。

CaP / Coding Agent

- 理解自然语言目标
- 执行排序、循环和条件判断
- 组合已有感知与操作 primitive
- 在测试时生成任务程序

传统 TAMP

- 绑定对象与目标位姿
- 选择左右机械臂和 handoff
- 检查工作空间与 IK
- 把宏动作降低为安全轨迹

控制器与验证器

- 50 Hz 仿真与 PID 控制
- 夹爪、升降和笛卡尔运动
- 终态几何误差判定
- 失败原因返回 Agent

分工原则： LLM 决定“做什么和如何组合”， TAMP/ 控制器保证“是否可达以及怎样稳定执行”。



本次新增的关键点是 **Live PolicyVM**：Python 控制流不再只记录静态动作列表，而是在每个宏动作执行后继续运行，因此 `robot.observe()` 能读取真实的新状态。

```
def policy(robot):  
    for obj, label in route:  
        robot.navigate()  
        obs = robot.observe()  
        slot = find_slot(obs['bins'],  
label)  
        robot.grasp('right', obj)  
        robot.release_at('right', slot)  
        robot.park('right')
```

在线执行的三个变化

1

动作已经作用于 MuJoCo

下一行代码看到的是新物理状态

2

目标在执行时绑定

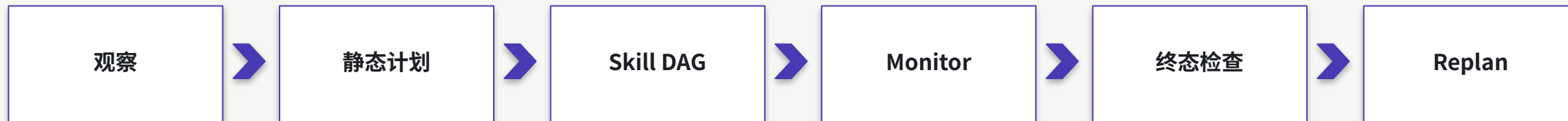
桶标签变化后重新查找物理槽位

3

失败可由程序处理

返回值和观察可以驱动条件与循环

仍受 AST、API、动作数和工作空间约束。



当前本地复现包含

- workflow DAG 与依赖验证
- skill 级监控事件
- 视觉差异形式的 spatial memory
- 终态不满足后的重规划

本实验中的限制

- DAG 在本轮开始前已经固定
- 动作均可达时不会中途触发失败
- 语义目标变化只在轮末验证时暴露
- 因此下一轮才会更新剩余动作

本地架构适配，不包含上游学习策略、VLA 权重和原始硬件系统。



当前本地复现包含

- target / policy 数据契约
- 严格 action bridge
- session 生命周期与 preflight
- heartbeat 和 watchdog recovery

本实验中的限制

- 适配器保证动作边界，不改写语义计划
- 一个 action chunk 内目标位置固定
- watchdog 未收到工具失败时不会打断
- 下一 session round 才读取新标签

本地架构适配，不包含上游学习策略、VLA 权重和原始硬件系统。

三个方块依次进入当前显示 5、4、3 的桶。

每次成功释放后，五个可见数字整体轮换一个物理槽位。变化发生在规划之后，无法从初始观察提前确定。

成功判据

所有方块位于释放当时显示目标数字的物理桶；容差 0.05 m

控制变量

- 同一 MuJoCo 场景与 seeds 411–413
- 同一三相机语义感知
- 同一 grasp / release / park 宏动作
- 同一 TAMP、IK、PID 与物理仿真
- 三者均获得准确对象和初始标签语义

主要指标：最终成功率、首策略成功率、规划轮数、基础动作数和终态误差。

依次将蓝、红、绿方块放入释放时显示 5、4、3 的桶。



蓝色方块

当前 5 号桶



红色方块

当前 4 号桶



绿色方块

当前 3 号桶

seed 411 的标签变化

初始	[5, 1, 3, 2, 4]
蓝色释放后	[1, 3, 2, 4, 5]
红色释放后	[3, 2, 4, 5, 1]

判定的是“释放当时的数字”

目标不是固定物理槽位。若计划只在开始时把数字转换成槽位，第一次释放后，后续目标位置就会过期。

自然语言任务：把蓝、红、绿三个方块依次放入当前的 5、4、3 号桶；每放下一个，桶编号改变一次。

为什么 Holo / Phy 需要三轮

CaP

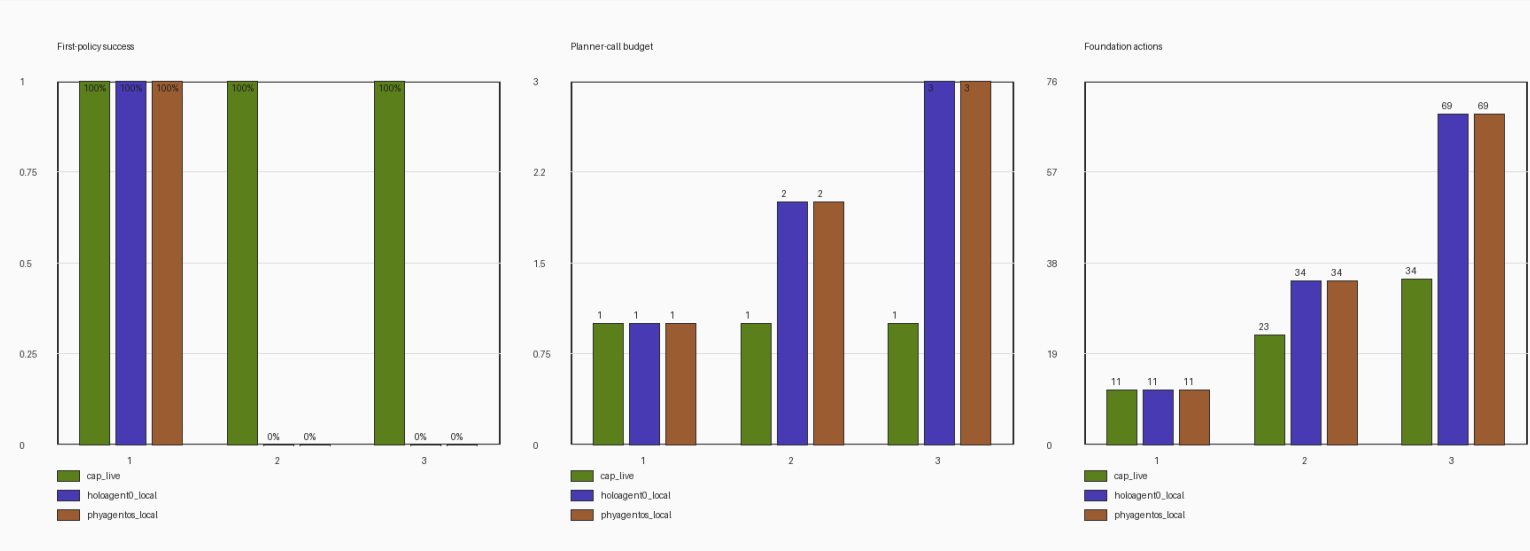


每次释放后立即观察新标签，三个动作均使用当前目标。

Holo / Phy



静态动作块中只有第一个目标仍然有效；轮末检查后再修正剩余对象。



最终成功率

三者 100%

三决策首策略

CaP 100% · 其余 0%

三决策规划次数

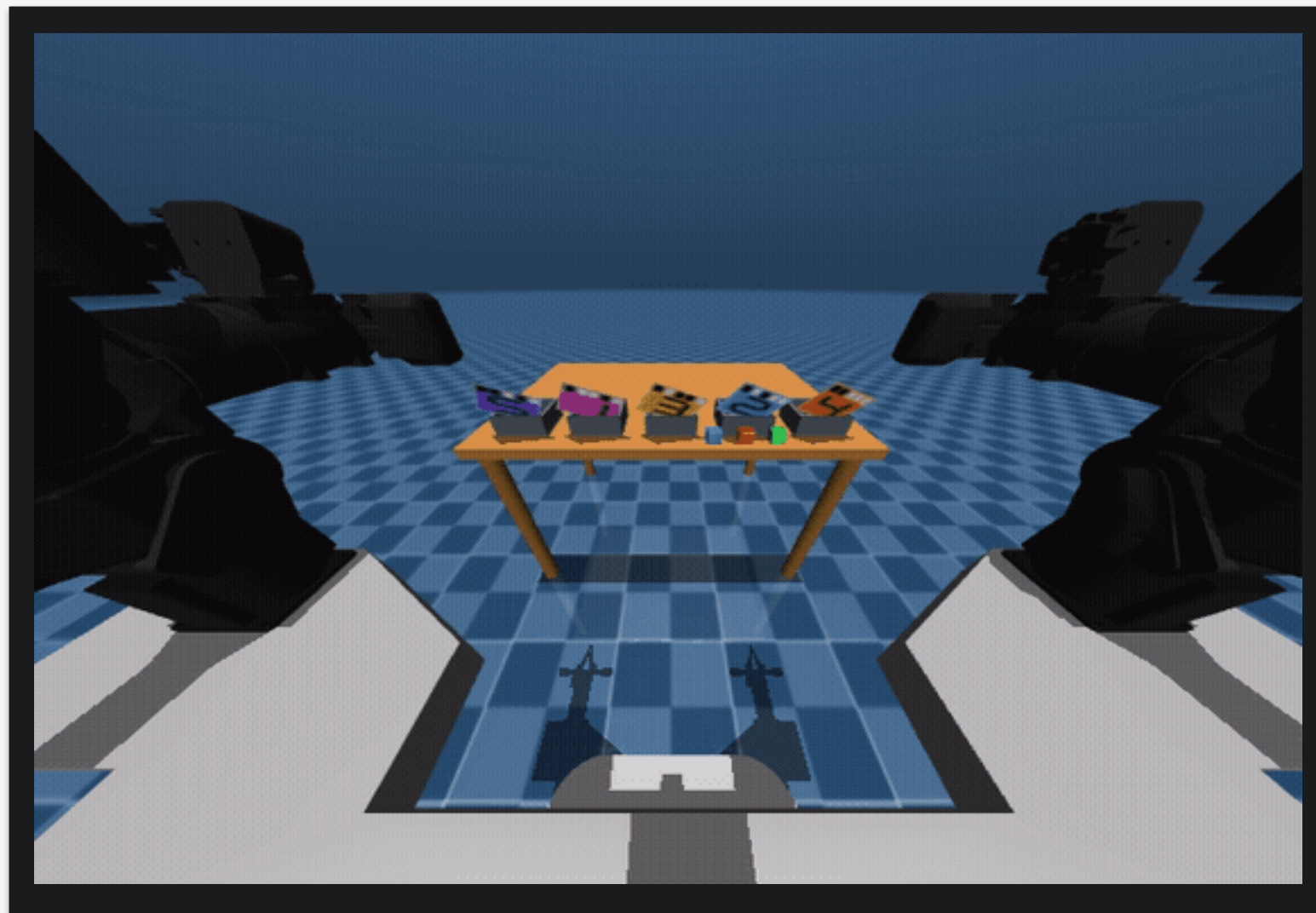
1 / 3 / 3

三决策动作数

34.3 / 68.7 / 68.7

CaP：一次策略内完成三次决策

10 · 真实运行



三决策 · seed 411

任务：蓝→5，红→4，绿→3

首策略成功 100%

规划 1 次

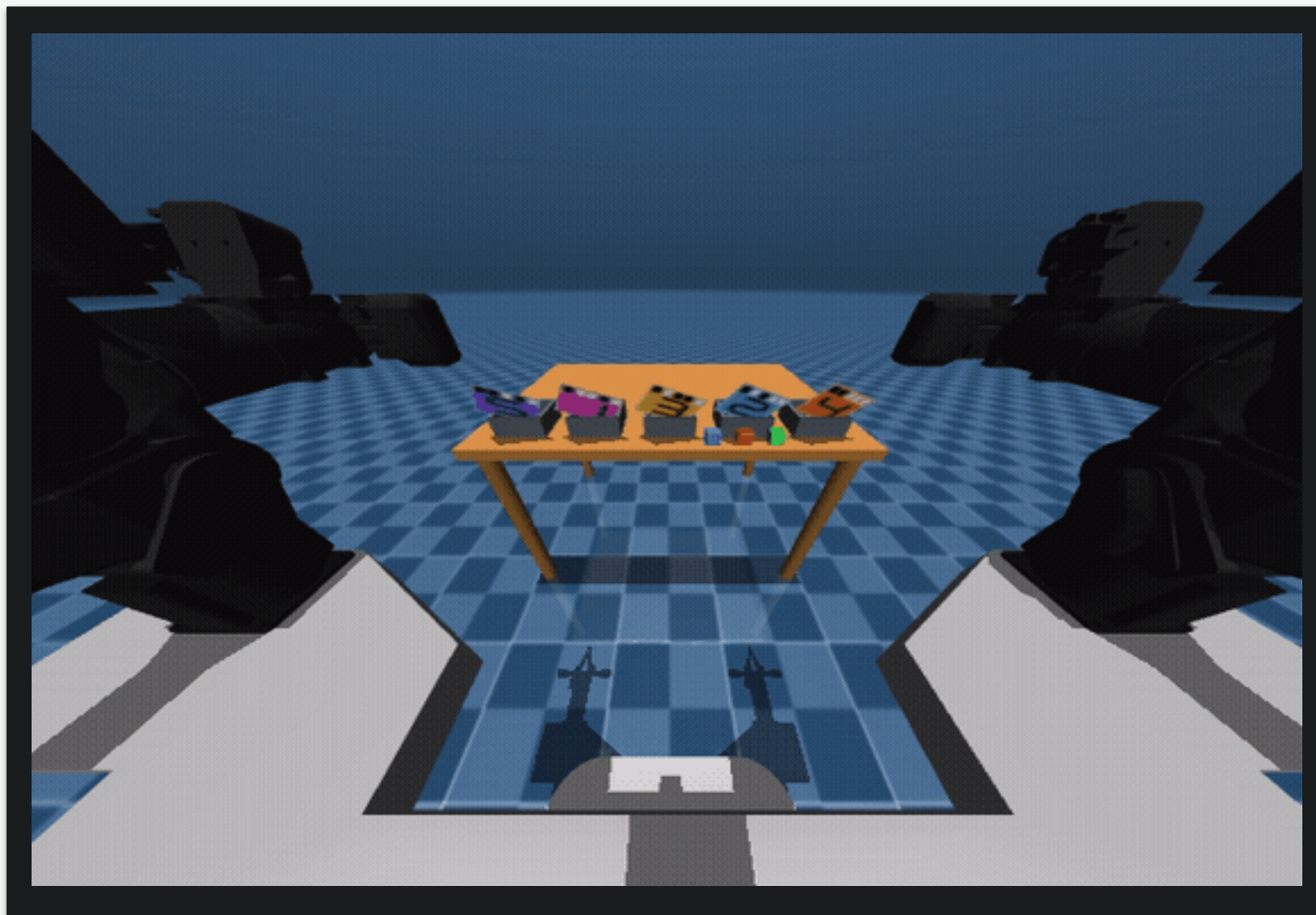
平均 34.33 个基础动作

程序在每次抓取前重新读取桶标签，
因此后续目标不会过期。

放映模式下 GIF 自动播放

HoloAgent-0 Local : DAG 执行后重规划

10 · 真实运行



三决策 · seed 411

任务：蓝→5，红→4，绿→3

首策略成功 0%

规划 3 次

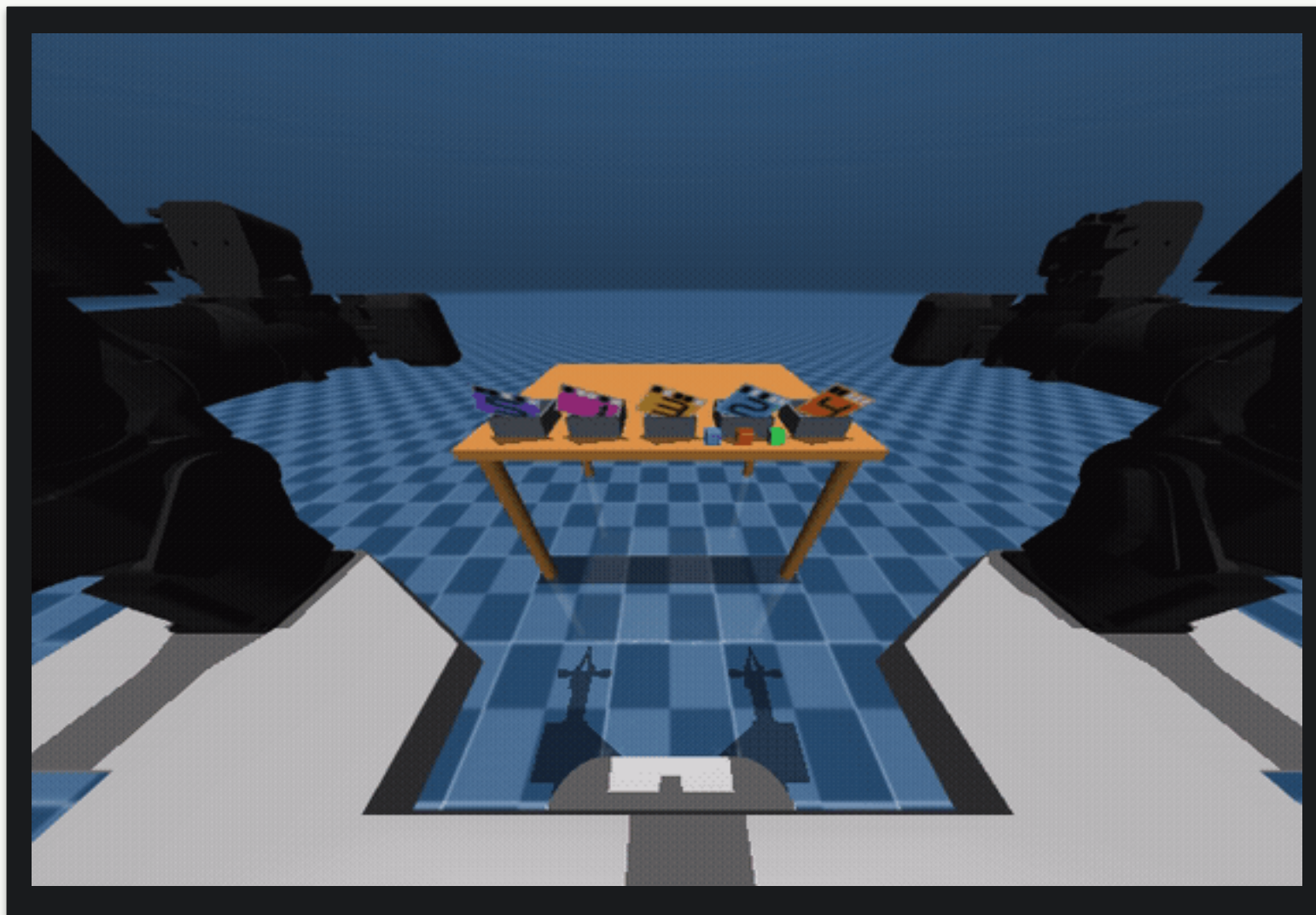
平均 68.67 个基础动作

workflow DAG 能监控并恢复任务，
但本轮静态动作块不会因语义变化
而中途改写。

放映模式下 GIF 自动播放

PhyAgentOS Local : Session 内重复下发动作块

10 · 真实运行



三决策 · seed 411

任务：蓝→5，红→4，绿→3

首策略成功 0%

规划 3 次

平均 68.67 个基础动作

adapter、bridge 和 watchdog
保证执行边界，但仍需下一轮观察
来更新动作块。

放映模式下 GIF 自动播放

01

Live PolicyVM

把 sandboxed Python 从静态动作记录器改成动作与控制流交错执行。

02

动态视觉任务

标签在动作完成后变化，终态判据按释放时可见标签计算。

03

三运行时统一协议

CaP、Holo workflow、Phy session 共用感知、TAMP 和控制器。

04

可复现实验资产

27 个 episode、JSON、结果图、MP4、GIF、PPT 和自动化测试。

贡献集中在 Harness 与执行架构；当前没有进行 VLA 或基础模型参数训练。

当新任务是已有感知和控制 primitive 的动态组合时，
CaP 的在线程序控制流能减少重规划和重复动作。

本实验支持

- 更高的首策略完成率
- 更少的 planning rounds
- 更少的重复机械动作
- 在线绑定动作后的新信息

本实验不支持

- CaP 在所有 OOD 上都更强
- 官方 Holo / Phy 性能排名
- 新的底层物理技能
- VLA 或 AgenticRL 训练结论

下一步实验

- 接入真实 LLM 请求延迟
- 空间探索与遮挡任务
- 多设备 / 多机器人调度
- 用 rollout 奖励优化 Harness

准确表述：CaP 更擅长 compositional zero-shot generalization ，而不是所有意义上的 zero-shot generalization 。